

Table des matières

Exercices 05 — CTE et récursivité	1
Questions courtes	1
Questions pratiques	2
Diagnostics	3
Questions à développer	5
Mises en situation	5

Exercices 05 — CTE et récursivité

Comment travailler ces exercices. Répondez à chaque énoncé seul, puis vérifiez. Les questions courtes vérifient que le vocabulaire est en place ; les pratiques vous font écrire du SQL, tantôt sur le transactionnel, tantôt sur l'entrepôt ; les diagnostics vous montrent du code qui cloche ; les mises en situation vous demandent de choisir une approche et de la défendre. Comptez cinq heures pour y répondre, le corrigé non compris.

Les repères de terrain. Un énoncé marqué [OLTP] se traite dans PostgreSQL sur Adventureworks (schémas sales, production). Un énoncé marqué [OLAP] se traite dans DuckDB sur `adventureworksdw.duckdb`, ouvert par `USE dbo;`. Sans marque, la question vaut pour les deux.

Le corrigé est dans `Corrige_05.md`. Répondez à l'énoncé en entier avant de l'ouvrir : c'est en butant sur une question qu'on découvre ce qu'on avait lu sans le comprendre.

Questions courtes

1. La définition. En une phrase, qu'est-ce qu'une CTE, et combien de temps existe-t-elle ?
2. Le mot-clé. Où s'écrit le `WITH` dans une requête, et combien peut-on en écrire pour une même requête, quel que soit le nombre de CTE ?
3. Le chaînage. Comment enchaîne-t-on trois CTE, et une CTE peut-elle lire celle qui la suit ?
4. L'astuce de débogage. Vous avez un pipeline de quatre CTE et le résultat final est faux. Quelle manipulation en une ligne vous permet d'inspecter n'importe quelle étape ?
5. Les trois outils. Remplissez de tête : sous-requête, CTE et vue, quelle est la portée de chacune, laquelle persiste, et quand choisir chacune ?
6. Réutiliser. Peut-on citer deux fois la même CTE dans le `SELECT` final ? Et une sous-requête, peut-on la citer deux fois sans la réécrire ?

7. Les trois morceaux. Nommez les trois parties d'une CTE récursive, dans l'ordre où on les écrit.
 8. UNION ALL ou UNION. Pourquoi écrit-on UNION ALL entre l'ancre et la partie récursive, et que changerait un UNION simple ?
 9. La condition d'arrêt. Qu'est-ce qui fait qu'une CTE récursive s'arrête ? Donnez les deux formes que prend cette condition.
 10. Le mot-clé RECURSIVE. Sur quel moteur est-il obligatoire, sur lequel est-il facultatif, et que faut-il écrire pour être portable ?
 11. Le message qui trompe. Sur PostgreSQL, vous oubliez RECURSIVE. Quel genre d'erreur obtenez-vous, et pourquoi ce message n'oriente-t-il pas vers la vraie cause ?
 12. La série générée. À quoi sert une série de valeurs fabriquée sans table ? Donnez l'usage principal en analytique.
 13. Combler les trous. Décrivez en trois étapes le mécanisme qui fait réapparaître les mois sans vente dans un rapport mensuel.
 14. [OLTP] Le raccourci. Quelle fonction de PostgreSQL remplace la CTE récursive pour générer une série d'entiers, et pourquoi apprend-on quand même la CTE ?
 15. [OLAP] Le raccourci d'en face. Qu'en est-il sur l'entrepôt, et quelle seconde forme DuckDB offre-t-il ? Attention à la borne.
 16. La boucle sans fin. Que se passe-t-il si la condition d'arrêt manque, sur chacun des deux moteurs de la formation ? Quelle parade se code, et pourquoi ne peut-on pas s'en remettre au moteur ?
 17. La matérialisation. Qu'est-ce qui a changé pour les CTE à PostgreSQL 12, et quel mot-clé permet de revenir au comportement d'avant ?
 18. Le mythe. « Une CTE, c'est plus rapide qu'une sous-requête. » Vrai, faux, ça dépend ? Justifiez en une phrase.
1. [OLAP] L'ancre d'un organigramme. Quelle condition attrape la racine dans DimEmployee, et pourquoi cette colonne est-elle nulle sur une seule ligne ?
 2. Le niveau. Comment obtient-on la profondeur hiérarchique de chaque ligne, sans jamais compter les tours de récursion à la main ?
 3. Le chemin. Comment construit-on une colonne qui affiche la chaîne complète depuis la racine ? Quel piège de type guette sur cette colonne ?
 4. [OLTP] La nomenclature. Dans production.billofmaterials, quelles sont les deux colonnes qui font de cette table un arbre, et vers quelle table pointent-elles toutes les deux ?
 5. [OLTP] La colonne de validité. Pourquoi filtre-t-on sur enddate IS NULL dans les deux morceaux de la récursion, et pas seulement dans l'ancre ?
 6. Les deux terrains. L'entrepôt vous donne un organigramme à plat, la base transactionnelle un arbre avec des dates de validité. Qu'est-ce que cet écart vous apprend sur le travail d'alimentation d'un entrepôt ?

Questions pratiques

Écrivez la requête complète avant de regarder. Quand un énoncé existe en deux versions, faites les deux et comparez-les : c'est là que le contraste entre les deux mondes devient concret.

P1. [OLAP] La CTE de lisibilité. Reprenez le « top 3 des produits par sous-catégorie » du Cours 04, mais sans QUALIFY : calculez le rang dans une CTE, puis filtrez à l'extérieur. C'est la version portable du QUALIFY.

P2. Les carrés. Avec une CTE récursive, générez les nombres de 1 à 10 et leur carré. Cette requête doit tourner à l'identique des deux côtés.

P3. [OLAP] L'effectif par niveau. À partir de l'organigramme, affichez combien d'employés se trouvent à chaque niveau hiérarchique.

P4. [OLAP] Le chemin complet. Reprenez l'organigramme et ajoutez une colonne `chemin` qui affiche la chaîne de commandement, façon Ken Sanchez > Brian Welcker >

P5. [OLTP] L'effectif par niveau, l'autre voie. Sur la nomenclature du produit 999, affichez combien de composants se trouvent à chaque niveau. Comparez la forme de l'arbre avec celle de l'organigramme.

P6. [OLTP] Le chemin complet, l'autre voie. Sur la même nomenclature, ajoutez une colonne `chemin` qui affiche la suite des noms de composants depuis le vélo. Attention au type de la colonne construite par concaténation.

P7. Le pipeline à trois étages. Écrivez, sur votre terrain préféré, une requête à trois CTE chaînées : les ventes par mois, puis la moyenne de ces mois, puis les seuls mois qui dépassent cette moyenne d'au moins 20 pour cent. Une CTE par idée, pas une de plus.

P8. [OLAP] L'année sans trous. Reprenez le rapport mensuel de 2010 avec les douze mois, et ajoutez une colonne qui indique par « oui » ou « non » si le mois a eu des ventes. Le COALESCE ne suffit pas ici, dites pourquoi.

P9. [OLTP] Le calendrier de la première année. Faites la même chose sur la première année de votre portage, en repérant l'année par une requête plutôt qu'en l'écrivant en dur.

P10. (avancé) La récursion bornée. Reprenez une des deux hiérarchies et ajoutez une double sécurité : une borne de profondeur explicite, et un compteur qui vous dit à quel tour la récursion s'est arrêtée. Dites ensuite ce que vous feriez de cette information dans du code de production.

P11. (avancé) Les feuilles de l'arbre. Sur la nomenclature du produit 999, ne gardez que les composants qui n'ont eux-mêmes aucun composant, autrement dit les pièces élémentaires. Deux écritures sont possibles, l'une qui interroge de nouveau la table de nomenclature, l'autre qui reste à l'intérieur du résultat de la CTE. Donnez les deux et dites laquelle vous préférez.

P12. (avancé) Le cumul des quantités. Toujours sur la nomenclature du produit 999, calculez pour chaque composant la quantité réellement nécessaire à la fabrication d'un vélo, c'est-à-dire le produit des quantités le long du chemin depuis la racine. Dites pourquoi une simple somme des `perassemblyqty` donnerait un résultat faux.

Diagnosics

On vous montre du code qui ne fait pas ce que son auteur croit. Dites ce qui cloche, pourquoi, et donnez le correctif.

E1. Un analyste veut les mois au-dessus de la moyenne. Il écrit, sur l'entrepôt :

```
WITH mensuel AS (  
    SELECT d.CalendarYear AS annee, d.MonthNumberOfYear AS mois,  
           SUM(f.SalesAmount) AS ventes  
    FROM dbo.FactInternetSales AS f  
    JOIN dbo.DimDate AS d ON d.DateKey = f.OrderDateKey  
    GROUP BY d.CalendarYear, d.MonthNumberOfYear  
)  
SELECT annee, mois, ventes  
FROM mensuel  
WHERE ventes > AVG(ventes);
```

Le moteur refuse la requête. Expliquez la vraie raison du refus, dites pourquoi l'auteur a cru que ça passerait, et donnez deux corrections : une avec une seconde CTE, une avec une fonction de fenêtrage du Cours 04.

E2. Une développeuse déroule la nomenclature sur PostgreSQL :

```
WITH nomenclature AS (  
    SELECT componentid, 1 AS niveau  
    FROM production.billofmaterials  
    WHERE productassemblyid = 999  
    UNION ALL  
    SELECT b.componentid, n.niveau + 1  
    FROM production.billofmaterials AS b  
    JOIN nomenclature AS n ON b.productassemblyid = n.componentid  
)  
SELECT * FROM nomenclature;
```

Elle obtient ERROR: relation "nomenclature" does not exist. Elle en conclut que sa CTE est mal nommée et la renomme trois fois. Dites ce qui cloche vraiment, pourquoi le message l'égare, et donnez le correctif. Signalez au passage un second défaut de cette requête, indépendant du premier.

E3. Un rapport doit montrer les douze mois de l'année, y compris ceux sans vente :

```
WITH RECURSIVE mois AS (  
    SELECT 1 AS m  
    UNION ALL  
    SELECT m + 1 FROM mois WHERE m < 12  
)  
SELECT mois.m, SUM(f.SalesAmount) AS ventes  
FROM dbo.FactInternetSales AS f  
JOIN dbo.DimDate AS d ON d.DateKey = f.OrderDateKey  
JOIN mois ON mois.m = d.MonthNumberOfYear
```

```
WHERE d.CalendarYear = 2010
GROUP BY mois.m
ORDER BY mois.m;
```

Le résultat ne contient toujours qu'une ligne. L'auteur ne comprend pas, puisqu'il a bien généré les douze mois. Expliquez, puis corrigez.

Questions à développer

Un court paragraphe de raisonnement, pas un mot juste.

A. La lisibilité est un critère technique. On dit souvent qu'une CTE ne sert qu'à « rendre le code joli », et donc que c'est du confort. Défendez la position inverse : expliquez en quoi la lisibilité d'une requête a des conséquences mesurables sur la qualité du travail, en vous appuyant sur des situations concrètes comme la relecture par un collègue, le débogage à six mois, ou la reprise d'une requête par quelqu'un d'autre. Terminez en disant où se situe la limite, c'est-à-dire à partir de quand découper en CTE nuit plutôt qu'il n'aide.

B. Récursion et confiance. Une CTE récursive est le seul outil de ce cours qui peut faire tomber un serveur. Expliquez le mécanisme du danger, puis décrivez la discipline que vous adopteriez avant de laisser une requête récursive tourner sur une base de production. Distinguez ce qui relève du code, ce qui relève des garde-fous du moteur, et ce qui relève de la connaissance des données.

C. L'arbre et l'entrepôt. Vous avez déroulé un organigramme sur l'entrepôt et une nomenclature sur le transactionnel. Expliquez, à quelqu'un qui bâtit un entrepôt, quelles décisions se prennent au moment d'y faire entrer une hiérarchie : que garde-t-on, que met-on à plat, que perd-on. Prenez position sur le cas des dates de validité, et dites ce que vous en feriez.

Mises en situation

On décrit une entreprise et une question métier. À vous de choisir l'approche, de l'écrire, et surtout de la justifier. Il y a plusieurs bonnes réponses, mais il y a des réponses indéfendables.

M1. Menuiserie Sainte-Adèle, le coût réel d'une commande. L'atelier fabrique des meubles sur mesure. Chaque meuble a une nomenclature à plusieurs niveaux, et chaque pièce élémentaire porte un coût unitaire dans une table de référence. On vous demande le coût de revient matière d'un meuble donné, c'est-à-dire la somme des coûts de toutes ses pièces élémentaires, chacune multipliée par la quantité réellement nécessaire compte tenu de sa place dans l'arbre.

Décrivez la requête : où va la récursion, comment se propage la quantité, et à quel moment se fait la multiplication par le coût. Le point difficile est la quantité cumulée le long du chemin : expliquez pourquoi une somme ne convient pas, et ce qui convient. Terminez en disant ce que vous faites si un composant apparaît à deux endroits de l'arbre du même meuble.

M2. Réseau de cliniques Rive-Sud, l'organigramme qui bouge. Le réseau compte 900 employés répartis sur onze établissements, avec une hiérarchie qui change souvent, et une table du personnel qui porte une colonne `superviseur_id` et deux colonnes de validité, `debut` et `fin`. La direction veut trois choses : l'organigramme au 1er janvier de l'an dernier, l'organigramme d'aujourd'hui, et la liste des personnes dont le niveau hiérarchique a changé entre les deux dates.

Écrivez le raisonnement complet. Précisez comment une date se glisse dans une CTE récursive, et pourquoi il vaut mieux la passer une fois plutôt que de la répéter dans les deux morceaux. Dites ensuite ce que vous faites des personnes arrivées ou parties entre les deux dates, qui n'existent que d'un côté. Terminez sur le point qui décide de tout : cette requête doit-elle vivre dans l'application ou dans un entrepôt, et qu'est-ce qui tranche ?

M3. (avancé) Coopérative de transport Chaudière, la tournée qui se referme. La coopérative modélise son réseau comme une table d'arêtes, *depart* et *arrivee*, entre points de collecte. On vous demande tous les trajets possibles entre deux points donnés, avec le nombre d'étapes. Le réseau contient des boucles : certains points se rejoignent dans les deux sens.

Écrivez la stratégie complète. Le cœur du problème est que la récursion, sur un graphe cyclique, ne s'arrête pas d'elle-même : décrivez précisément les deux mécanismes de protection, celui qui borne la profondeur et celui qui empêche de repasser par un point déjà visité, et dites pourquoi le second est le seul qui soit vraiment correct. Comparez ensuite le coût de cette requête sur un entrepôt et sur une base transactionnelle, puis répondez à la question qui fâche : est-ce que le SQL est le bon outil pour ce problème, et si non, à partir de quelle taille de réseau changez-vous d'avis ?